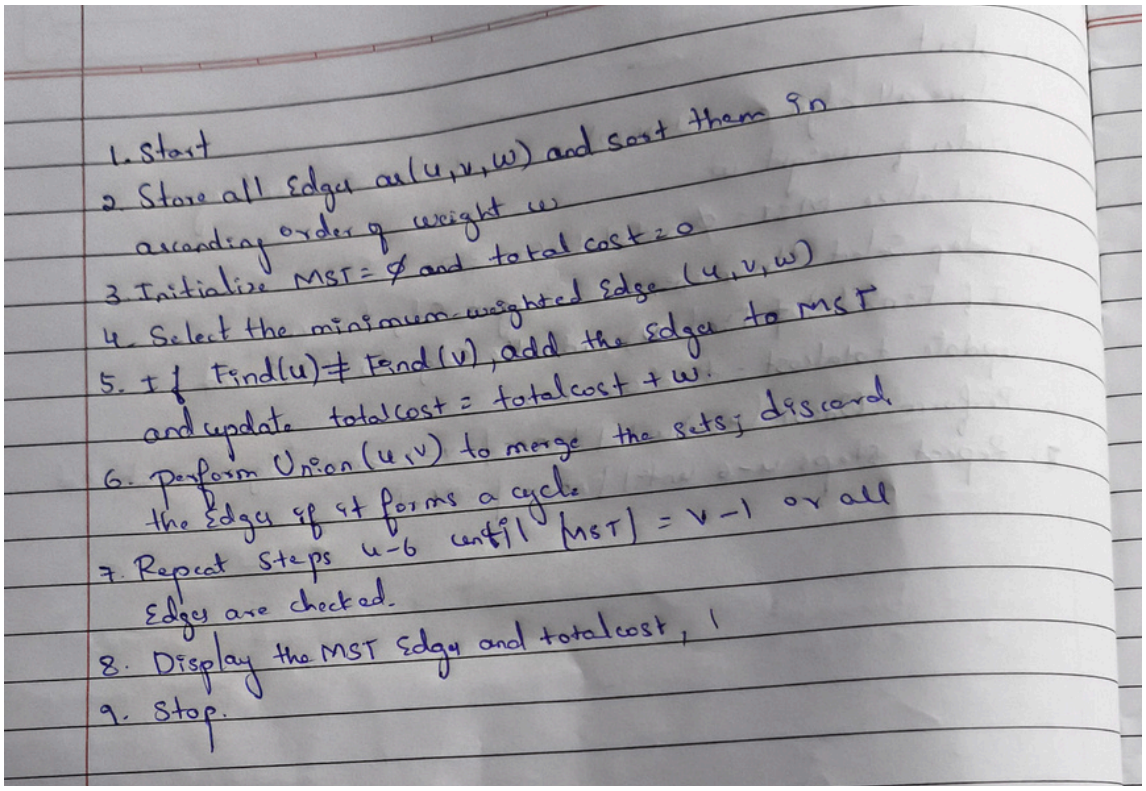
Practical: 10

Aim: Implement Kruskal's algorithm.

Algorithm:



program code:

```
#include <iostream>
#include <stack>
#include <algorithm>
using namespace std;

#define MAX 20

struct Edge
{
   int u, v, weight;
};

int parent[MAX];

// Find parent of a vertex
int findParent(int x)
{
   while (parent[x] != x)
      x = parent[x];

   return x;
}

// Check whether adding an edge creates a cycle
bool hasCycle(int u, int v)
{
   return findParent(u) == findParent(v);
}

// Join two sets
void unionSet(int u, int v)
{
   int parentU = findParent(u);
   int parentV = findParent(v);

   parent[parentV] = parentU;
}

int main()
{
```

```
int n, e;

cout<<"Enternumberofvertices: ";
cin >> n;

cout<<"Enternumberofedges: ";
cin >> e;

Edge edges[MAX];

cout<<"Enteredges(source destination weight):\n";

for (int i = 0; i < e; i++)
{
   cin >> edges[i].u
      >> edges[i].v
      >> edges[i].weight;
}

//Sortedgesaccordingtoweight
sort(edges,edges+e,[](Edge a, Edge b)
{
   returna.weight<b.weight;
});

// Initialize parent
for(inti =0;i <n;i++)
   parent[i] = i;

stack<Edge> mst;

int totalCost= 0;
int count = 0;

// Kruskal's algorithm
for(inti=0;i<e&&count < n - 1; i++)
{
   int u = edges[i].u;
   int v = edges[i].v;

   if (!hasCycle(u, v))
   {
```

```
            mst.push(edges[i]);
            unionSet(u, v);

            totalCost += edges[i].weight;
            count++;
        }
    }

    cout<<"\nEdges in Minimum Spanning Tree:\n";

    //DisplayMST
    while(!mst.empty())
    {
        Edgecurrent = mst.top();
        mst.pop();

        cout<<current.u << " - "
            <<current.v
            <<"Weight: "
            <<current.weight << endl;
    }

    cout<<"\nTotal cost of Minimum Spanning Tree = "
        <<totalCost << endl;

    return 0;
}
```

Output:

```
Enter number of vertices: 4
Enter number of edges: 5
Enter edges (source destination weight):
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4

Edges in Minimum Spanning Tree:
0 - 1  Weight: 10
0 - 3  Weight: 5
2 - 3  Weight: 4

Total cost of Minimum Spanning Tree = 19
```